

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина:

«DevOps»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 2

Выполнил:

Айметов Андрей Алексеевич,

студент группы N3146



(подпись)

Проверила:

Мигулаева Татьяна Алексеевна,

инженер факультета прикладной информатики

(подпись)

Санкт-Петербург

2026г

СОДЕРЖАНИЕ

1. ЗАДАНИЕ	3
2. ПРИМЕРЫ РАБОТЫ ПРОГРАММ	4
3. ИСХОДНЫЙ КОД ПРОГРАММЫ.....	7
4. ПОЧЕМУ НЕЛМ УДОБНЕЕ.....	11

1. ЗАДАНИЕ

1 часть

Поднять kubernetes кластер локально (например minikube), в нём развернуть свой сервис, используя 2-3 ресурса kubernetes. В идеале разворачивать кодом из yaml файлов одной командой запуска. Показать работоспособность сервиса.

(сервис любой из своих не опенсорсных, вывод “hello world” в браузер тоже подойдёт)

2 часть

1. Создать helm chart на основе обычной 3 лабы
2. Задеплоить его в кластер
3. Поменять что-то в сервисе, задеплоить новую версию при помощи апгрейда релиза
4. В отчете приложить скрины всего процесса, все использованные файлы, а также привести три причины, по которым использовать хелм удобнее чем классический деплой через кubernetes манифесты

2. ПРИМЕРЫ РАБОТЫ ПРОГРАММ

1 часть

```
(sanwerde@colorfulkali)-[~/Рабочий стол/Лабы/DevOps/Lab2]
$ sudo usermod -aG docker $USER && newgrp docker
[sudo] пароль для sanwerde:
(sanwerde@colorfulkali)-[~/Рабочий стол/Лабы/DevOps/Lab2]
$ minikube start
🐳 minikube v1.38.1 на Debian kali-rolling
🌟 Используется драйвер docker на основе существующего профиля
👍 Starting "minikube" primary control-plane node in "minikube" cluster
📡 Pulling base image v0.0.50 ...
🔄 Перезагружается существующий docker container для "minikube" ...
🔧 Подготавливается Kubernetes v1.35.1 на Docker 29.2.1 ...
🔍 Компоненты Kubernetes проверяются ...
  ▪ Используется образ gcr.io/k8s-minikube/storage-provisioner:v5
🌟 Включенные дополнения: storage-provisioner, default-storageclass

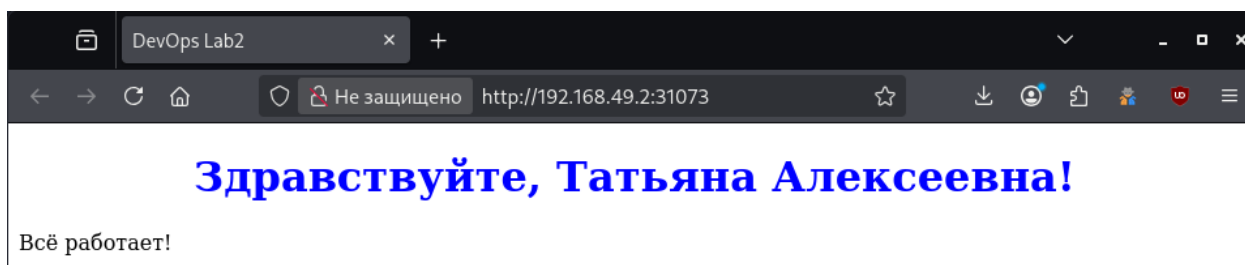
! /usr/local/bin/kubectl is version 1.31.0, which may have incompatibilities
with Kubernetes 1.35.1.
  ▪ Want kubectl v1.35.1? Try 'minikube kubectl -- get pods -A'
🎉 Готово! kubectl настроен для использования кластера "minikube" и "default"
пространства имён по умолчанию

(sanwerde@colorfulkali)-[~/Рабочий стол/Лабы/DevOps/Lab2]
$ kubectl apply -f ./SK8
namespace/my-app created
configmap/hello created
deployment.apps/web-server created
service/web-svc created

(sanwerde@colorfulkali)-[~/Рабочий стол/Лабы/DevOps/Lab2]
$ minikube service web-svc -n my-app
```

NAMESPACE	NAME	TARGET PORT	URL
my-app	web-svc	80	http://192.168.49.2:31073

🎉 Opening service my-app/web-svc in default browser...

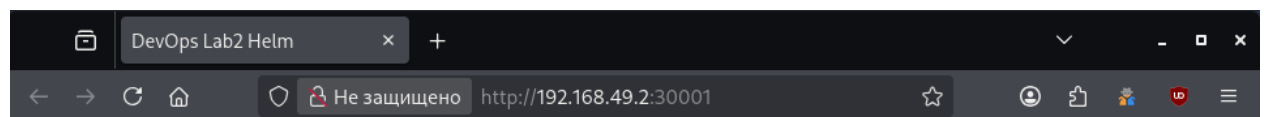


```
(sanwerde@colorfulkali)-[~/Рабочий стол/Лабы/DevOps/Lab2]
$ helm install my-release ./my-app-chart -n my-app --create-namespace
NAME: my-release
LAST DEPLOYED: Sat Apr 11 23:44:20 2026
NAMESPACE: my-app
STATUS: deployed
REVISION: 1
DESCRIPTION: Install complete
TEST SUITE: None
```

```
(sanwerde@colorfulkali)-[~/Рабочий стол/Лабы/DevOps/Lab2]
$ minikube service web-svc -n my-app
```

NAMESPACE	NAME	TARGET PORT	URL
my-app	web-svc	80	http://192.168.49.2:30001

🌈 Opening service my-app/web-svc in default browser...



Здравствуйте, Татьяна Алексеевна!

Всё работает на Helm!

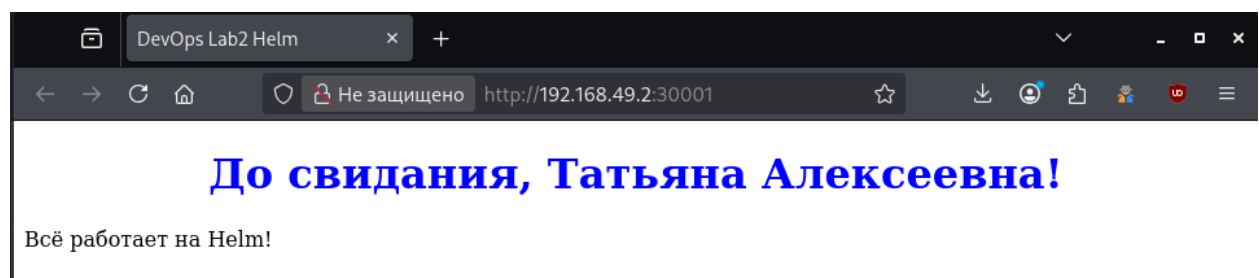
```
(sanwerde@colorfulkali)-[~/Рабочий стол/Лабы/DevOps/Lab2]
$ helm upgrade my-release ./my-app-chart -n my-app
Release "my-release" has been upgraded. Happy Helming!
NAME: my-release
LAST DEPLOYED: Sat Apr 11 23:49:41 2026
NAMESPACE: my-app
STATUS: deployed
REVISION: 2
DESCRIPTION: Upgrade complete
TEST SUITE: None

(sanwerde@colorfulkali)-[~/Рабочий стол/Лабы/DevOps/Lab2]
$ kubectl rollout restart deployment/web-server -n my-app
deployment.apps/web-server restarted

(sanwerde@colorfulkali)-[~/Рабочий стол/Лабы/DevOps/Lab2]
$ minikube service web-svc -n my-app
```

NAMESPACE	NAME	TARGET PORT	URL
my-app	web-svc	80	http://192.168.49.2:30001

🌐 Opening service my-app/web-svc in default browser...



3. ИСХОДНЫЙ КОД ПРОГРАММЫ

1 часть

3.1. 01-namespace.yaml

```
kind: Namespace
metadata:
  name: my-app
```

3.2. 02-configmap.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: hello
  namespace: my-app
data:
  index.html: |
    <!DOCTYPE html>
    <html>
    <head>
      <meta charset="UTF-8">
      <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
      <title>DevOps Lab2</title>
    </head>
    <body>
      <h1 style="color: blue; text-align: center;">Здравствуйте, Татьяна
Алексеевна!</h1>
      <p>Всё работает!</p>
    </body>
    </html>
```

3.3. 03-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-server
  namespace: my-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: web-server
  template:
    metadata:
      labels:
        app: web-server
```

```

spec:
  containers:
    - name: nginx
      image: nginx:1.27-alpine
      ports:
        - containerPort: 80
      volumeMounts:
        - name: html-volume
          mountPath: /usr/share/nginx/html/index.html
          subPath: index.html
      resources:
        requests:
          memory: "64Mi"
          cpu: "100m"
        limits:
          memory: "128Mi"
          cpu: "250m"
  volumes:
    - name: html-volume
      configMap:
        name: hello

```

3.4. 04-service.yaml

```

apiVersion: v1
kind: Service
metadata:
  name: web-svc
  namespace: my-app
spec:
  type: NodePort
  selector:
    app: web-server
  ports:
    - port: 80
      targetPort: 80
      protocol: TCP

```


3.5. configmap.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: hello-config
  namespace: my-app
data:
  index.html: |
    <!DOCTYPE html>
    <html>
    <head>
      <meta charset="UTF-8">
      <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
      <title>{{ .Values.configMap.title }}</title>
    </head>
    <body>
      <h1 style="color: blue; text-align: center;">{{ .Values.configMap.greeting }}</h1>
      <p>{{ .Values.configMap.message }}</p>
    </body>
    </html>
```

3.6. deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-server
  namespace: my-app
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: web-server
  template:
    metadata:
      labels:
        app: web-server
    spec:
      containers:
        - name: nginx
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          ports:
            - containerPort: 80
```

```
volumeMounts:
  - name: html-volume
    mountPath: /usr/share/nginx/html/index.html
    subPath: index.html
volumes:
  - name: html-volume
    configMap:
      name: hello-config
```

3.7. service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: web-svc
  namespace: my-app
spec:
  type: {{ .Values.service.type }}
  selector:
    app: web-server
  ports:
    - port: {{ .Values.service.port }}
      targetPort: 80
      nodePort: {{ .Values.service.nodePort }}
      protocol: TCP
```

values

3.8. values.yaml

```
replicaCount: 2
image:
  repository: nginx
  tag: "1.27-alpine"
configMap:
  greeting: "До свидания, Татьяна Алексеевна!"
  message: "Всё работает на Helm!"
  title: "DevOps Lab2 Helm"
service:
  type: NodePort
  port: 80
  nodePort: 30001
```

4. ПОЧЕМУ HELM УДОБНЕЕ

Параметризация (DRY — Don't Repeat Yourself)

Можно поменять один раз значение в `values.yaml`, и оно поменяется везде.

Управление жизненным циклом и откаты (Versioning & Rollback)

Helm хранит историю релизов (`helm history`). Каждая команда `upgrade` — это новая ревизия. Если деплой сломался, команда `helm rollback` мгновенно возвращает кластер к предыдущему рабочему состоянию.

Безопасный Dry-Run и встроенная валидация

Helm `lint` проверяет чарт на best practices и синтаксис. `helm upgrade --dry-run --debug` рендерит все манифесты, показывает diff изменений и ничего не применяет. Ошибки ловятся до контакта с кластером.